

Data Control Language

Linguagem de Controle de Dados (DCL)

- É um subconjunto da linguagem SQL, voltado ao controle de permissões e segurança no banco de dados, ou seja, define quem pode acessar, alterar ou manipular determinados objetos (tabelas, views, procedures, etc.).

- **Comandos**

  CREATE ROLE

  ALTER ROLE

  DROP ROLE

  GRANT

  REVOKE

CREATE ROLE

- Gerenciar acesso e permissões é essencial em qualquer sistema de banco de dados. O PostgreSQL oferece um modelo flexível e poderoso baseado em **usuários e funções (roles)**.

- Contudo, no PostgreSQL, usuários e roles (funções) não são entidades completamente separadas.

- Tecnicamente, o PostgreSQL trata tudo como uma função . A diferença está em se uma função tem privilégios de login ou não.

CREATE ROLE

- **Role**: Um conceito mais amplo que pode representar usuários, grupos ou conjuntos de privilégios. Funções podem ser concedidas a outras funções, permitindo um gerenciamento flexível de permissões.

```
CREATE ROLE readonly;
CREATE ROLE admin_role;
CREATE ROLE sales_user;
```

CREATE ROLE

- **Usuário**: Uma função com privilégios de login. Pode se autenticar e executar comandos conforme seus privilégios.

```
CREATE ROLE john LOGIN PASSWORD 'secure_password';
```

```
CREATE ROLE leo
WITH
  LOGIN
  PASSWORD 'secure_password'
  VALID UNTIL '2025-12-31'
  CONNECTION LIMIT 5;
```

GRANT

- O GRANT no PostgreSQL (e em outros bancos de dados SQL) é o comando utilizado para conceder permissões a um usuário ou role

- Também pode ser usado para atribuir um role a um usuário, algo como atribuir perfis de usuários

- Ele define quem pode realizar determinadas operações (como SELECT, INSERT, UPDATE, DELETE, etc.) sobre objetos como tabelas, views, funções ou esquemas.

- SINTAXE: GRANT permissao ON nivel TO role_name

GRANT

GRANT SELECT ON produtos **TO** readonly;

GRANT readonly **TO** john;

GRANT SELECT, **INSERT**, **UPDATE ON** clientes **TO** john;

-- Concede permissão para acessar (USAGE) e criar objetos (CREATE) dentro do schema public.
-- OBS: USAGE não lê nem lista dados; apenas permite utilizar o schema.
GRANT USAGE, **CREATE ON SCHEMA** vendas **TO** sales_user;

GRANT ALL PRIVILEGES **ON ALL** TABLES **IN SCHEMA** public **TO** admin_role;

GRANT ALL PRIVILEGES **ON** DATABASE meu_banco **TO** admin_role;

GRANT

- Importante: conceder privilégios em nível de DATABASE (ex.: GRANT ALL ON DATABASE ...) não concede automaticamente permissões sobre as tabelas, views ou sequências existentes.

- Os privilégios dos níveis DATABASE, TABLE, SCHEMA e SEQUENCE devem ser atribuídos separadamente.

- Ou seja, os níveis de permissão são independentes: um usuário pode ter acesso ao banco, mas continuar sem permissão para consultar ou alterar tabelas.

REVOKE

- É utilizado para revogar (retirar) permissões anteriormente concedidas a usuários ou roles no banco de dados.

- Serve para restringir acessos ou ajustar privilégios, garantindo maior controle e segurança sobre os objetos do banco.

- Sintaxe básica:

REVOKE <PRIVILÉGIOS> ON <OBJETO> FROM <ROLE>;

REVOKE

REVOKE INSERT, UPDATE ON clientes **FROM** john**;**

REVOKE ALL PRIVILEGES ON produtos **FROM** readonly**;**

REVOKE SELECT ON vendas **FROM** operador**;**

REVOKE ALL PRIVILEGES ON DATABASE meu_banco **FROM** admin_role;

ALTER ROLE

- O comando ALTER ROLE é utilizado para modificar propriedades de uma role (usuário ou grupo) já existente no PostgreSQL.

- Ele pode alterar permissões gerais do usuário, como:

LOGIN ou NOLOGIN – permitir ou impedir acesso ao banco
SUPERUSER – conceder privilégios administrativos totais
CREATEDB – permitir criar bancos de dados
CREATEROLE – permitir criar novas roles
PASSWORD – alterar a senha de acesso
VALID UNTIL – definir data de expiração

ALTER ROLE

- **Importante:** ALTER ROLE não altera privilégios sobre objetos existentes (tabelas, esquemas, bancos). Para isso, ainda é necessário usar GRANT / REVOKE.

- **Sintaxe básica:**

ALTER ROLE nome_da_role [WITH] opção1 opção2 ... ;

REVOKE

```
-- Alterar a senha de uma role existente.
ALTER ROLE leo WITH PASSWORD 'nova_senha';

-- Eleva o usuário 'leo' ao nível de SUPERUSER.
-- Um SUPERUSER no PostgreSQL ignora todas as regras de permissões (GRANT/REVOKE), passando a ter acesso irrestrito a todos os bancos, schemas e tabelas.
ALTER ROLE leo WITH SUPERUSER;

-- Alterar validade da role.
ALTER ROLE leo VALID UNTIL '2026-12-31';

-- Ajustar limite de conexões permitidas.
ALTER ROLE leo CONNECTION LIMIT 10;
```

DROP ROLE

- O comando DROP ROLE é usado para excluir uma role (usuário ou grupo) do PostgreSQL.

- Você só pode remover uma role se:

- Ela não tiver privilégios ativos
- Ela não estiver atribuída a outra role

DROP ROLE sales_user;

PG_ROLE

- pg_roles é uma view que lista todas as roles existentes no PostgreSQL, incluindo usuários e grupos.

- Ela mostra informações como:

- Nome da role (rolname)
- Se pode fazer login (rolcanlogin)
- Se é superusuário (rolsuper)
- Se pode criar bancos (rolcreatedb)
- Se pode criar outras roles (rolcreaterole)
- Data de expiração (rolvaliduntil)

```
SELECT * FROM pg_roles;
```

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software